

Program 6: Jenkins CI Pipeline — Complete Explanation from Zero

First — What is Jenkins in Simple Words?

Imagine you are a developer. Every time you write new code, you have to manually:

1. Open command prompt
2. Run `mvn compile`
3. Run `mvn test`
4. Run `mvn package`
5. Run the JAR file
6. Check if everything works

This is boring and time consuming. What if you have to do this 20 times a day every time someone changes the code?

Jenkins solves this problem.

Jenkins watches your GitHub repository. The moment anyone pushes new code, Jenkins automatically:

1. Downloads the new code from GitHub
2. Compiles it
3. Tests it
4. Packages it into a JAR
5. Runs it
6. Tells you if it succeeded or failed

You don't have to do anything manually! This is called **Continuous Integration (CI)**.

What is CI/CD? (Must Know for Viva)

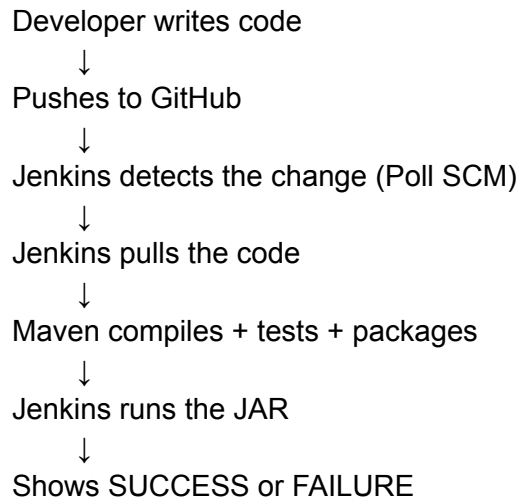
CI = Continuous Integration

- Every time new code is pushed to GitHub, it is automatically built and tested
- "Continuous" means it happens automatically every time, not just once
- Benefits: bugs are found quickly, team always has a working build

CD = Continuous Deployment

- After the build succeeds, the application is automatically deployed to a server
- In Program 8 we will add Ansible to do this part

The Pipeline stages are:



What is a Freestyle Project in Jenkins?

Jenkins has different job types. **Freestyle project** is the simplest one where you configure everything through buttons and forms in the Jenkins website (no scripting needed). It is the best starting point for beginners.

Before Starting — What You Need

You need these installed:

1. **Java JDK** — Jenkins runs on Java
2. **Maven** — to build the project
3. **Jenkins** — the main tool
4. **Git** — to push code to GitHub
5. **GitHub account** — to store your code online

To check Jenkins is running, open your browser and go to:

<http://localhost:8080>

You should see the Jenkins login page. If you don't see it, Jenkins is not running. Tell me and I will explain how to start it.

PART 1: Prepare the Maven Project and Push to GitHub

This part is done on your computer before touching Jenkins.

STEP 1: We Will Use the Maven Project from Program 4

We are going to use the `maven-example` project we already created in Program 4. It is inside:

`C:\Users\YourName\Desktop\MAVENTOGRADLEPROJECT\maven-example`

If you did not do Program 4, create a fresh Maven project:

```
cd desktop
mkdir MAVENTOGRADLEPROJECT
cd MAVENTOGRADLEPROJECT
mvn archetype:generate -DgroupId=com.example -DartifactId=maven-example
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
cd maven-example
```

STEP 2: Make Sure pom.xml is Correct

Open:

`notepad pom.xml`

Delete everything and paste this. This pom.xml is slightly different from before — it includes the `maven-jar-plugin` which is **very important** for Jenkins to be able to run the JAR:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
```

<http://maven.apache.org/xsd/maven-4.0.0.xsd>>

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId>

<artifactId>maven-example</artifactId>

<version>1.0-SNAPSHOT</version>

<dependencies>

<dependency>

<groupId>junit</groupId>

<artifactId>junit</artifactId>

<version>4.12</version>

<scope>test</scope>

</dependency>

</dependencies>

<build>

<pluginManagement>

<plugins>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-compiler-plugin</artifactId>

<version>3.0.2</version>

<configuration>

<source>1.8</source>

<target>1.8</target>

</configuration>

</plugin>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-jar-plugin</artifactId>

<version>3.2.2</version>

<configuration>

<archive>

<manifest>

<addClasspath>true</addClasspath>

<mainClass>com.example.App</mainClass>

</manifest>

</archive>

</configuration>

</plugin>

</plugins>

</pluginManagement>

```
</build>
```

```
</project>
```

Press **CTRL + S** to save.

Why is **maven-jar-plugin** important?

Without it, when Jenkins tries to run the JAR using **java -jar**, it gives this error:

no main manifest attribute

This error means Java does not know which class has the **main()** method. The **maven-jar-plugin** with **<mainClass>com.example.App</mainClass>** fixes this by writing the main class name into the JAR's MANIFEST file.

STEP 3: Make Sure App.java is Correct

Open:

notepad src\main\java\com\example\App.java

Delete everything and paste:

```
package com.example;
```

```
public class App {
```

```
    public static void main(String[] args) {
        System.out.println("Hello, Maven");
        System.out.println("This is the simple realworld example....");

        int a = 5;
        int b = 10;
        System.out.println("Sum of " + a + " and " + b + " is " + sum(a, b));
    }
```

```
    public static int sum(int x, int y) {
        return x + y;
    }
```

```
}
```

Press **CTRL + S** to save.

STEP 4: Create a GitHub Repository

Now you need to put your project on GitHub so Jenkins can access it.

1. Open your browser and go to <https://github.com>
2. Log in to your account
3. Click the **+** button at top right → click **New repository**
4. Give it a name: **SAMPLEJENKINSPROJECT**
5. Keep it **Public**
6. Do NOT check "Add a README file"
7. Click **Create repository**

GitHub will show you a page with repository URL. Copy that URL. It looks like:

<https://github.com/yourusername/SAMPLEJENKINSPROJECT.git>

STEP 5: Push Your Project to GitHub Using Git Bash

Open **Git Bash** (not Command Prompt — Git Bash is different). You can find it by right clicking on your Desktop and selecting "Git Bash Here" or searching in Start menu.

Now navigate to your maven-example folder:

```
cd desktop
```

```
cd MAVENTOGRADEPROJECT
```

```
cd maven-example
```

Now run these Git commands one by one:

Command 1 — Initialize Git in this folder:

```
git init
```

This creates a hidden `.git` folder that tracks all changes. Output:

Initialized empty Git repository in ...maven-example/.git/

Command 2 — Add all files to staging area:

```
git add .
```

The `.` means "add ALL files". Staging means "mark these files as ready to be committed".

Command 3 — Commit the files:

```
git commit -m "first commit"
```

`-m` means message. "first commit" is the message describing what you changed. Output shows all files that were committed.

Command 4 — Rename branch to main:

```
git branch -M main
```

By default Git creates a branch called "master". This command renames it to "main" which is the modern standard GitHub uses.

Command 5 — Connect your local folder to GitHub:

```
git remote add origin https://github.com/yourusername/SAMPLEJENKINSPROJECT.git
```

Replace `yourusername` with your actual GitHub username. This command tells Git — "the online copy of this project lives at this URL".

Command 6 — Push (upload) your files to GitHub:

```
git push -u origin main
```

This uploads all your files to GitHub. It will ask for your GitHub username and password (or personal access token).

Note about password: GitHub no longer accepts regular passwords. You need a **Personal Access Token (PAT)**. To create one:

1. Go to GitHub → click your profile picture → Settings
2. Scroll down → click "Developer settings"
3. Click "Personal access tokens" → "Tokens (classic)"
4. Click "Generate new token"
5. Give it a name, select "repo" checkbox, click Generate
6. Copy the token — use this as your password when Git asks

After pushing, go to <https://github.com/yourusername/SAMPLEJENKINSPROJECT> in your browser. You should see all your project files there. This means upload was successful.

PART 2: Set Up Jenkins

STEP 6: Open Jenkins

Open your browser and go to:

`http://localhost:8080`

You will see the Jenkins login page with a friendly robot/butler logo.

Enter your username and password and click **Sign in**.

You will see the Jenkins Dashboard — a page showing "Welcome to Jenkins!" with options like "Create a job".

STEP 7: Install Maven Plugin in Jenkins

Before creating a job, we need to make sure Jenkins has the Maven plugin installed.

1. On the left side panel, click "**Manage Jenkins**"
2. Click "**Plugins**"
3. Click "**Available plugins**" tab
4. In the search box type: `maven integration`
5. You will see "**Pipeline Maven Integration**" in the list
6. Check the checkbox next to it
7. Click "**Install**" button
8. Wait for it to install. You will see a progress bar
9. Once done, go back to Dashboard by clicking "Dashboard" at top left

Why do we need this plugin? Jenkins by itself does not know how to run Maven commands. This plugin adds Maven support to Jenkins.

STEP 8: Configure Maven in Jenkins

Jenkins also needs to know where Maven is installed on your computer.

1. Click **"Manage Jenkins"**
 2. Click **"Tools"**
 3. Scroll down to find **"Maven installations"** section
 4. Click **"Add Maven"**
 5. Give it a name: `maven`
 6. Check **"Install automatically"** OR if Maven is already installed, uncheck it and give the path where Maven is installed (e.g. `C:\apache-maven-3.9.0`)
 7. Scroll down and click **"Save"**
-

STEP 9: Create a New Jenkins Job

Now we create the actual CI job.

1. On Jenkins Dashboard, click **"New Item"** on the left side
2. You will see a text box saying "Enter an item name"
3. Type: `JENKINSPROJECT`
4. Below that you will see different project types. Click on **"Freestyle project"**
5. Click **"OK"** at the bottom

You will now see the configuration page for your new job. This is where we set up everything.

STEP 10: Configure General Settings

You are now on the configuration page. You will see several sections on the left: General, Source Code Management, Triggers, Environment, Build Steps, Post-build Actions.

In the **General** section:

- You can add a description if you want (optional)
- Check **"Discard old builds"** to save disk space
- Under Strategy select "Log Rotation"
- Max # of builds to keep: type `5`

STEP 11: Configure Source Code Management

This is where we tell Jenkins where our code lives.

1. Click on **"Source Code Management"** on the left OR scroll down to find that section
2. You will see two radio buttons: "None" and "Git"
3. Click on **"Git"**
4. A form will appear asking for Repository URL
5. In "Repository URL" paste your GitHub URL:

<https://github.com/yourusername/SAMPLEJENKINSPROJECT.git>

6. Under "Credentials" click **"Add" → "Jenkins"**
7. A popup appears:
 - Kind: select "Username with password"
 - Username: your GitHub username
 - Password: your GitHub Personal Access Token (the one you created earlier)
 - ID: type **GITHUB CREDENTIALS**
 - Click "Add"
8. Now in the Credentials dropdown, select the credentials you just added
9. Under "Branches to build" in "Branch Specifier" type:

`*/main`

This tells Jenkins to watch the `main` branch.

STEP 12: Configure Build Triggers

This is where we tell Jenkins WHEN to build.

1. Click **"Triggers"** on the left OR scroll to Triggers section
2. Check the checkbox **"Poll SCM"**
3. A "Schedule" text box appears. Type:

`H/5 * * * *`

What does `H/5 * * * *` mean? (Important for Viva)

This is a cron expression — a way to schedule tasks:

- First * = minutes
- Second * = hours
- Third * = day of month
- Fourth * = month
- Fifth * = day of week

H/5 * * * * means "check every 5 minutes". Jenkins will check your GitHub every 5 minutes. If it finds new code, it will automatically start a build.

The H instead of a number means Jenkins picks a random starting point to avoid all jobs running at exactly the same time.

STEP 13: Configure Build Steps

This is the most important part — what should Jenkins actually DO when it builds.

1. Click **"Build Steps"** on the left OR scroll to that section
2. Click **"Add build step"** button
3. From the dropdown, select **"Invoke top-level Maven targets"**
4. A form appears:
 - Maven Version: select **maven** (the one we configured in Step 8)
 - Goals: type exactly this:

clean compile test package

5.

What does **clean compile test package** mean?

- **clean** — delete all previously compiled files (start fresh)
 - **compile** — compile the Java source code
 - **test** — run all unit tests
 - **package** — create the JAR file
5. Now click **"Add build step"** again
 6. This time select **"Execute Windows batch command"**
 7. In the Command box type:

```
java -cp target/maven-example-1.0-SNAPSHOT.jar com.example.App
```

This runs the JAR file that Maven just created and prints the output.

STEP 14: Save the Configuration

Scroll down to the bottom and click the blue **"Save"** button.

You will be taken to the project page showing "JENKINSPROJECT".

STEP 15: Build the Project

Now let's manually trigger a build to test everything.

1. On the left side panel, click **"Build Now"**
2. You will see a build appear in the "Build History" section at the bottom left
3. It shows a progress circle/ball — blue means success, red means failure
4. Click on the build number (like **#1**)
5. Click on **"Console Output"** on the left

You will see all the logs of what Jenkins did. Scroll to the bottom and you should see:

Hello, Maven

This is the simple realworld example....

Sum of 5 and 10 is 15

Finished: SUCCESS

BUILD SUCCESS means your CI pipeline is working!

STEP 16: Test That Automation Works

Now let's verify that Jenkins automatically builds when you push new code.

1. Go back to Git Bash
2. Make a small change to App.java — maybe add a print statement
3. Run:

```
git add .
```

```
git commit -m "small change"
```

```
git push origin main
```

4. Wait about 5 minutes (Jenkins checks every 5 minutes)

5. Go to Jenkins and check Build History
6. You should see a new build #2 started automatically!

This proves CI is working — Jenkins detected the code change and built automatically without you clicking "Build Now".

Complete Summary of All Steps

— ON YOUR COMPUTER —

1. Prepare Maven project with correct pom.xml and App.java
2. Create GitHub repository called SAMPLEJENKINSPROJECT
3. Open Git Bash in maven-example folder
4. git init
5. git add .
6. git commit -m "first commit"
7. git branch -M main
8. git remote add origin https://github.com/yourusername/SAMPLEJENKINSPROJECT.git
9. git push -u origin main
10. Verify files appear on GitHub

— IN JENKINS (http://localhost:8080) —

11. Manage Jenkins → Plugins → Install "Pipeline Maven Integration"
12. Manage Jenkins → Tools → Add Maven installation
13. New Item → name: JENKINSPROJECT → Freestyle project → OK
14. Source Code Management → Git → paste GitHub URL → add credentials → branch: */main
15. Triggers → Poll SCM → schedule: H/5 * * * *
16. Build Steps → Invoke top-level Maven targets → Goals: clean compile test package
17. Build Steps → Execute Windows batch command → java -cp target/maven-example-1.0-SNAPSHOT.jar com.example.App
18. Save
19. Build Now → check Console Output → verify SUCCESS